

# ShiftSwap: A Shift Schedule and Substitute Management System for Small Part-Time Workplaces

소규모 매장 아르바이트 근무표·대타 관리 시스템

Table 1. Team 7

| Name               | Student ID | Affiliation                                             | E-mail              |
|--------------------|------------|---------------------------------------------------------|---------------------|
| Hyewon Kim (김혜원)   | 24102028   | Dept. of Industrial Engineering, ITM Program, SeoulTech | (to be provided)    |
| Hyoungdo Kim (김형도) | 18102075   | Dept. of Industrial Engineering, ITM Program, SeoulTech | zloganway@gmail.com |
| Minkyung Kim (김민경) | 23102004   | Dept. of Industrial Engineering, ITM Program, SeoulTech | (to be provided)    |

Course: Software Engineering, Seoul National University of Science and Technology (SeoulTech)

Submitted to: jihjim@gmail.com

Date: 2 October 2026

## Contents

1. Need for the proposed system
  2. Project goals
  3. Description of the problem
    - 3.1 Reference store and current process
    - 3.2 Problems P1–P3
  4. Scope
  5. Main functions of the target system
    - 5.1 Actors
    - 5.2 Function overview
    - 5.3 Core functions in detail
    - 5.4 Supporting functions
    - 5.5 Data to be stored
  6. Development approach
  7. Work breakdown structure
  8. Project schedule
  9. Roles of team members
  10. Risks and mitigation
- References

## 1. Need for the proposed system

Small stores such as cafés, convenience stores and restaurants often run with fewer than ten part-time workers. Each worker has fixed weekly shifts, for example every Monday from 18:00 to 23:00. Workers are often students, so exams and other duties regularly make them miss a shift. When this happens, somebody else has to cover the shift.

In most such stores the substitute is found by telephone or chat message. The owner contacts other workers one at a time until one of them agrees. The owner then edits the schedule spreadsheet by hand. The swap is not only a schedule change. It also changes the weekly hours of two workers, and in Korea weekly hours decide whether the weekly holiday allowance (주휴수당) is owed. At month end the owner has to reconstruct these changes to pay each worker correctly.

Korean pay rules for part-time work are detailed. The 2026 minimum hourly wage is KRW 10,320 [1, 2]. A worker whose weekly contractual hours are 15 or more and who attended every scheduled day is entitled to a paid weekly holiday [3, 5]. Overtime and night premiums apply only in workplaces with five or more regular employees [3]. The minimum wage may be reduced to 90 % during probation only under specific conditions [4]. An owner of a four-person café applies these rules from memory. Errors are found on payday or later.

General chat applications broadcast a message but do not know who is free, who answered first, or what a swap does to pay. Shift-scheduling products such as Shiftee, the work-manage-

ment features of Albamon and Alba Cheonguk, When I Work, Homebase and Deputy manage schedules and shift trades [10]. None of those we examined connects the approval of a substitution with the Korean holiday-allowance rule at the moment of approval. The owner therefore cannot see, before approving a swap, what the swap does to either worker's entitlement. The proposed system, ShiftSwap, closes this gap. It is a conventional transaction-processing business application, as the course recommends: it records shifts, requests, responses, approvals, attendance and payroll, and applies fixed business rules to each transaction.

## 2. Project goals

The overall goal is to deliver a working web application that replaces the telephone-and-spreadsheet process of a small store for three tasks: finding a substitute, keeping the schedule correct, and computing weekly holiday allowance and monthly pay. The goals below are stated so that each one can be checked at the end of the project.

**Table 2. Measurable project goals**

| ID | Goal                                                                         | Measure / acceptance check                                                                                                                                                                                                                                            | Problem |
|----|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| G1 | Remove the owner's one-to-one contacts when a substitute is needed.          | The owner makes 0 contacts per request (reference store today: 3–5). All eligible workers are notified in the same transaction that creates the request (FR-07). The owner's only action is one approve or reject decision.                                           | P1      |
| G2 | Let a candidate answer quickly.                                              | A candidate accepts a request in at most 2 clicks from the inbox (NFR-01).                                                                                                                                                                                            | P1      |
| G3 | Keep the schedule and weekly hours of both workers correct after every swap. | On approval, the shift is reassigned and the weekly summaries of both workers are recomputed with no manual step (FR-12). The approval screen shows both workers' hours and eligibility before and after the swap (FR-11).                                            | P2      |
| G4 | Apply the legal pay rules automatically and flag violations before payday.   | Business rules BR-06 to BR-11 are implemented as tested functions. Every business rule has at least one unit test (spec §12). The payroll screen warns when a wage is below the applicable minimum and blocks confirmation until the warning is acknowledged (FR-16). | P3      |
| G5 | Deliver a system of the size the course requires.                            | At least 5 core use cases (planned: 13, of which 9 are core) and at least 4 database tables (planned: 12).                                                                                                                                                            | –       |
| G6 | Deliver on time and reproducibly.                                            | Working system deployed at a public URL by Week 14 (2026-12-04, NFR-07). Re-running the final prompt in an empty folder produces a system that passes the same unit tests.                                                                                            | –       |

## 3. Description of the problem

### 3.1 Reference store and current process

The topic was proposed by Hyewon Kim from her own part-time job. Her workplace is our reference store. It is a small store with fewer than ten part-time workers, and the owner also acts as the manager. We selected this topic on 2026-09-27 after comparing seven candidate topics. The current process for a missed shift is as follows.

1. A worker learns that they cannot work one of their fixed shifts. They tell the owner by message or telephone.
2. The owner looks at the schedule and guesses who might be free. The owner contacts these workers one at a time. In the reference store this takes typically 3 to 5 contacts per request.
3. When a worker agrees, the owner edits the schedule spreadsheet by hand and tells both workers.
4. At month end the owner goes back through the spreadsheet. For each worker and each week the owner re-counts the hours, decides whether the weekly holiday allowance is owed, and computes the pay.

Each step is manual. No step keeps a record of who was asked, who answered, or when the owner agreed. The spreadsheet is the only record of the swap, and it holds the result, not the reason.

### 3.2 Problems P1–P3

**P1** Finding a substitute costs the owner repeated one-to-one contacts (reference store: typically 3–5 contacts per request).

**P2** The schedule, weekly hours and holiday-allowance eligibility of *two* workers change with every swap, and all recalculation is manual.

**P3** Legal pay rules (minimum wage, holiday allowance, 5-employee threshold for premiums, probation reduction) are applied from memory, so errors are not caught before payday.

P2 needs an example. Suppose Lee Seoyeon (fixed shifts Monday and Wednesday 18:00–23:00, 10 hours per week) gives her Wednesday shift to Choi Minh (fixed shifts 16 hours per week). Her scheduled hours for that week fall from 10 to 5. His rise from 16 to 21. His contractual hours stay at 16, so he remains eligible for the holiday allowance:  $16 / 40 \times 8 = 3.2$  hours, which is  $3.2 \times 10,320 = \text{KRW } 33,024$ . Her contractual hours stay at 10, so she is not eligible either way. The owner currently has to work this out by hand for every swap.

P3 has a further difficulty. The law does not say directly whether a shift that was handed over by agreement breaks the worker's "perfect attendance" for the week. Section 10 explains how the system treats this.

## 4. Scope

We classify every function by MoSCoW priority. *Must* functions form the first increment; *Should* functions form the second increment (Section 6). *Won't* items are outside this project.

**Table 3. Scope of the project**

| Priority | Included items                                                                                                                                                                                                                                             |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Must     | UC-01, UC-02, UC-03, UC-04, UC-05, UC-06, UC-08, UC-11, UC-13 and the in-app notification inbox (FR-21). Together they form a usable system: a store can find substitutes, approve swaps and see weekly holiday-allowance eligibility.                     |
| Should   | UC-07 attendance, UC-09 payroll with minimum-wage and probation checks, UC-10 settings, UC-12 cancellation.                                                                                                                                                |
| Won't    | KakaoTalk or SMS messaging (notifications are in-app only); the four major social insurances and tax deduction; annual leave and severance pay; bank transfer of wages; QR-code or fingerprint clock-in (attendance is self-reported and owner-confirmed). |

## 5. Main functions of the target system

### 5.1 Actors

The system has three actors. **A1 Worker** (primary) is a part-time employee who views their own schedule, registers availability, requests substitutes, responds to requests and records attendance. **A2 Owner** (primary) is the store owner, who also acts as manager: the owner registers workers and fixed schedules, approves or rejects substitutions, confirms attendance, reviews weekly summaries, confirms monthly payroll and edits settings. **A3 System Clock** (secondary) is a time-triggered actor that expires unanswered or unapproved requests at their deadline and closes weeks for the weekly summary; in the demo application it is driven by an adjustable demo clock.

### 5.2 Function overview

The system has 13 use cases. UC-01 to UC-09 are the nine main functions of this proposal. UC-10 to UC-13 support them. Table 5 lists each use case with its primary actor, the database tables it uses and its priority. Section 5.3 gives, for each main function, the trigger, input, processing with the business rules (BR) it applies, output and outcome.

**Table 5. Use cases of ShiftSwap**

| ID    | Use case                           | Primary actor | Main tables           | Priority |
|-------|------------------------------------|---------------|-----------------------|----------|
| UC-01 | Register worker and fixed schedule | Owner         | Worker, FixedSchedule | Must     |

| ID    | Use case                                         | Primary actor      | Main tables                                | Priority |
|-------|--------------------------------------------------|--------------------|--------------------------------------------|----------|
| UC-02 | Register availability                            | Worker             | Availability                               | Must     |
| UC-03 | Generate weekly schedule                         | Owner              | Shift                                      | Must     |
| UC-04 | Request substitute                               | Worker             | SubRequest, SubRequestTarget, Notification | Must     |
| UC-05 | Respond to substitute request (accept / decline) | Worker (candidate) | SubRequestTarget, SubRequest               | Must     |
| UC-06 | Approve or reject substitution                   | Owner              | SubRequest, Shift, WeeklySummary           | Must     |
| UC-07 | Record and confirm attendance                    | Worker, Owner      | Attendance, Shift                          | Should   |
| UC-08 | Compute weekly hours and holiday allowance       | Owner (System)     | WeeklySummary, Workplace                   | Must     |
| UC-09 | Generate and confirm monthly payroll             | Owner              | Payroll, MinimumWage, Worker               | Should   |
| UC-10 | Manage workplace settings and minimum wage table | Owner              | Workplace, MinimumWage                     | Should   |
| UC-11 | Expire overdue substitute request                | System Clock       | SubRequest, SubRequestTarget, Notification | Must     |
| UC-12 | Cancel own substitute request                    | Worker             | SubRequest, SubRequestTarget               | Should   |
| UC-13 | Sign in by role (demo account selection)         | Worker, Owner      | Worker                                     | Must     |

## 5.3 Core functions in detail

This section describes the nine main functions (F1–F9 = UC-01 to UC-09) in terms of input, processing and output. Examples use the demo store of the application, Dalbit Café (달빛카페), which has one owner (Park Jiyoung) and four workers.

### F1. Register worker and fixed schedule (UC-01)

**Trigger:** A new worker is hired or a contract changes. **Rules:** – **Tables:** Worker, FixedSchedule

**Input.** Name, phone, hourly wage in KRW, contract start and end date, optional probation end date, simple-labor flag and active flag (FR-01). One or more fixed weekly shifts, each given as weekday, start time and end time (FR-02).

**Processing.** The system rejects a fixed shift that overlaps another fixed shift of the same worker. The sum of a worker's fixed-shift hours is the worker's weekly *contractual hours*, which BR-06 uses later.

**Output.** The worker list and the worker's fixed schedule. Example: Choi Minhoo works Tuesday and Thursday 18:00–23:00 and Saturday 12:00–18:00, which gives 16 contractual hours per week.

## F2. Register availability (UC-02)

**Trigger:** A worker wants to offer time for substitute shifts. **Rules:** BR-01 (input) **Tables:** Availability

**Input.** Availability slots (weekday, start, end) during which the worker can take extra shifts (FR-03).

**Processing.** Slots are stored per worker. They are the second half of the candidate rule BR-01.

**Output.** The worker's own availability list. Example: Choi Minhoo is available Monday and Wednesday 17:00–23:00.

## F3. Generate weekly schedule (UC-03)

**Trigger:** The owner opens a week that has no shifts yet. **Rules:** BR-05 **Tables:** Shift

**Input.** A week, identified by its Monday date (BR-05: a week runs from Monday 00:00 to Sunday 24:00).

**Processing.** The system creates one shift per active fixed-schedule entry for that week. Generating the same week twice creates no duplicates (FR-04). The owner can then add, move or delete a single shift (FR-05).

**Output.** A weekly board with seven columns, Monday to Sunday. Each shift shows its time range and worker. A shift that was handed over shows the new worker and the original worker.

## F4. Request substitute (UC-04)

**Trigger:** A worker cannot work one of their future shifts. **Rules:** BR-01, BR-03 **Tables:** SubRequest, SubRequestTarget, Notification

**Input.** One of the requester's own future shifts, an optional reason and a response deadline (FR-06).

**Processing.** The request must satisfy BR-03: only the shift's current worker may request; the shift is SCHEDULED and starts in the future; at most one open request exists per shift; the deadline is after now and no later than the shift start. The system then selects candidates with BR-01. A worker is eligible if the worker is active, is not the requester, has no shift overlapping the requested shift on that date, and has an availability slot on that weekday that covers the whole shift. One target record and one inbox notification are created per candidate in the same transaction (FR-07). If nobody is eligible, the request is still stored and the owner is notified (FR-08).

**Output.** The list of notified candidates, or the message that no eligible worker exists. Example: Lee Seoyeon requests a substitute for Wednesday 2026-09-30 18:00–23:00 with the

reason “Midterm exam” and deadline 2026-09-29T21:00. Choi Minhø and Kang Doyun are both free and available on Wednesday evening, so both are notified.

## **F5. Respond to substitute request (UC-05)**

**Trigger:** A request arrives in a candidate's inbox. **Rules:** BR-02 **Tables:** SubRequestTarget, SubRequest

**Input.** Accept or decline, chosen directly on the inbox item (at most 2 clicks, NFR-01).

**Processing.** BR-02 “first acceptance wins”: an acceptance succeeds only if the request is still REQUESTED when the transaction runs. It sets the request to ACCEPTED, records the acceptor, and closes all other pending targets. The check runs inside one database transaction, so two acceptances can never both succeed (NFR-04).

**Output.** The requester and the owner are notified (FR-10). A later acceptance is refused with an explanatory message (FR-09): the candidate sees “Already taken by <name>”. Example: Choi Minhø accepts first; Kang Doyun's acceptance is refused.

## **F6. Approve or reject substitution (UC-06)**

**Trigger:** A request has been accepted. **Rules:** BR-04, BR-06, BR-07 **Tables:** SubRequest, Shift, WeeklySummary

**Input.** The owner's decision on an ACCEPTED request.

**Processing.** Before the decision the system shows a comparison for the requester and the acceptor: contractual hours, scheduled hours for the week and holiday-allowance eligibility, before and after the swap. Any change in eligibility is highlighted (FR-11). On approval (BR-04) the shift is assigned to the acceptor, the original worker is kept on the shift record, the request becomes APPROVED, and the weekly summaries of both workers are recomputed (F8). On rejection the shift stays unchanged and the request becomes REJECTED (FR-12).

**Output.** Notifications to both workers. The weekly board shows the handover marker. Example: in the approval panel, Choi Minhø's scheduled hours change from 16 to 21 and his eligibility stays “eligible”.

## **F7. Record and confirm attendance (UC-07)**

**Trigger:** A shift has ended. **Rules:** BR-06 (input) **Tables:** Attendance, Shift

**Input.** Actual clock-in and clock-out time, entered by the worker for a past shift; the owner's confirmation, or the owner's decision to mark the shift ABSENT (FR-13).

**Processing.** A confirmed record sets the shift to WORKED. An ABSENT shift breaks perfect attendance for that week (BR-06). No location data is collected (NFR-12).

**Output.** Attendance list per shift with its confirmation state.

## **F8. Compute weekly hours and holiday allowance (UC-08)**

**Trigger:** A swap is approved, attendance changes, or the owner opens a week. **Rules:** BR-05, BR-06, BR-07 **Tables:** WeeklySummary, Workplace

**Input.** Fixed schedules, shifts, attendance and the workplace policy for one week.

**Processing.** For each worker the system computes contractual, scheduled and actual hours and the perfect-attendance flag (FR-14). BR-06: the worker is eligible if weekly contractual hours are at least 15 and every shift of the week that the worker did not give away was WORKED. A shift given away by an approved substitution counts according to the workplace policy: EXCUSED (default, attendance not broken) or ABSENT (attendance broken). BR-07: holiday hours =  $\min(\text{contractual hours}, 40) / 40 \times 8$ , rounded to two decimals; holiday pay = hours  $\times$  hourly wage, rounded down to the won.

**Output.** A weekly summary table per worker with a short reason when a worker is not eligible, for example “contract hours below 15”.

## F9. Generate and confirm monthly payroll (UC-09)

**Trigger:** Month end. **Rules:** BR-08 to BR-11 **Tables:** Payroll, MinimumWage, Worker

**Input.** A calendar month.

**Processing.** BR-11: base pay = actual hours of WORKED shifts in the month  $\times$  wage; if attendance is not recorded yet, scheduled hours are used and the row is flagged “estimated”. Holiday pay is the sum over weeks whose Sunday falls in the month. BR-08: premium pay (+50 % for daily hours beyond 8, weekly hours beyond 40 and night hours 22:00–06:00) applies only when the workplace has five or more regular employees. BR-09: the system warns when the wage is below the minimum wage of that year. BR-10: the minimum may be reduced to 90 % only if the contract is at least one year (or open-ended), the date lies within three months of the contract start and not after the probation end, and the work is not simple labor.

**Output.** A draft payroll per worker with base pay, holiday pay, premium pay, total and flags. A minimum-wage warning must be acknowledged before the owner can confirm. Confirmed rows are read-only (FR-15, FR-16). Example: Kang Doyun earns KRW 10,000 per hour, below the 2026 minimum of KRW 10,320, so his row shows a warning.

## 5.4 Supporting functions

- **UC-10 Settings** (Owner; Workplace, MinimumWage). Trigger: store data, employee count, policy or legal wage changes. The owner edits the store name, the number of regular employees (drives BR-08), the substitution-attendance policy EXCUSED or ABSENT (drives BR-06) and the yearly minimum wage rows (2025: KRW 10,030; 2026: KRW 10,320) (FR-17).
- **UC-11 Expiry** (System Clock; SubRequest, SubRequestTarget, Notification). When the deadline of an open request passes, or its shift starts, the request becomes EXPIRED. Its targets are closed and the requester and owner are notified (FR-18, BR-12).
- **UC-12 Cancellation** (Worker; SubRequest, SubRequestTarget). A requester cancels their own request while it is REQUESTED or ACCEPTED (FR-19).
- **UC-13 Sign-in** (Worker, Owner; Worker). The user chooses a demo account; the interface shows only the menus of that role (FR-20). A worker never sees another worker's phone number or wage (NFR-11).
- **Notification inbox.** Every user has an in-app inbox with the newest notification first and an unread count (FR-21).

## 5.5 Data to be stored

The system stores its data in 12 relational tables, above the course minimum of four: Workplace, Worker, FixedSchedule, Availability, Shift, SubRequest, SubRequestTarget, Attendance, WeeklySummary, Payroll, MinimumWage and Notification. A substitute request moves through the states REQUESTED, ACCEPTED, APPROVED, REJECTED, EXPIRED and CANCELLED.

## 6. Development approach

**Life-cycle model.** We follow the *incremental model* with *prototyping* of the user interface (Chapter 1). Requirements and analysis models are written once for the whole system. Implementation is then delivered in two increments. Increment 1 contains all *Must* use cases and is itself a usable system: a store can find substitutes, approve swaps and see weekly holiday-allowance eligibility. Increment 2 adds the *Should* use cases: attendance, payroll, settings and cancellation.

We chose this model for three reasons. First, the deadline is fixed (Week 14), and the incremental model delivers a working system early, so a delay affects only the second increment. Second, the pay rules carry legal uncertainty; a working first increment lets the reference-store owner comment on real screens before the payroll part is built. Third, the waterfall model handles change poorly, and our requirements for the approval screen are likely to change after the owner sees it. A UI prototype of the key screens (weekly board, inbox, approval panel) will be shown to the owner and two co-workers during requirements validation. The prototype is built with the same tool chain as the final system, so it is evolved rather than thrown away.

**Implementation method.** The system is implemented by vibe coding with an AI coding assistant, as the assignment requires. The team writes a specification and a final prompt; the assistant generates the code. Each prompt version and its result is kept in a prompt log so that the final prompt reliably reproduces the system.

**Technology.** A static web application (HTML, CSS and JavaScript modules) with an embedded SQLite database compiled to WebAssembly (sql.js [9]). The data persists in the browser, so no server is needed. The application is hosted on GitHub Pages. Models follow UML 2.5 notation [8].

## 7. Work breakdown structure

The work breakdown structure (WBS) in Figure 1 has eight top-level packages that follow the course deliverables and the life-cycle phases: project management, proposal, requirements, design, implementation by vibe coding, testing, final report and presentation. Table 6 lists every work package with its deliverables and owner; Figure 1 follows the table. Owners are abbreviated as HW (Hyewon Kim), HD (Hyoungdo Kim) and MK (Minkyung Kim).

**Table 6. Work packages, deliverables and owners (level 3 items are listed in the deliverable column)**

| ID         | Work package                         | Deliverables                                                                                  | Owner  |
|------------|--------------------------------------|-----------------------------------------------------------------------------------------------|--------|
| <b>1</b>   | <b>Project management</b>            |                                                                                               |        |
| <b>1.1</b> | Scheduling & WBS                     | 1.1.1 WBS and Gantt chart; 1.1.2 weekly milestone tracking                                    | HD     |
| <b>1.2</b> | Team coordination                    | 1.2.1 weekly meeting notes; 1.2.2 task list with owners                                       | HD     |
| <b>2</b>   | <b>Proposal</b>                      |                                                                                               |        |
| <b>2.1</b> | Topic selection                      | 2.1.1 comparison of seven candidate topics; 2.1.2 selected topic (2026-09-27)                 | HW     |
| <b>2.2</b> | Proposal document                    | 2.2.1 problem statement and scope (HW); 2.2.2 reviewed proposal PDF (HD)                      | HW, HD |
| <b>3</b>   | <b>Requirements (interim report)</b> |                                                                                               |        |
| <b>3.1</b> | Elicitation                          | 3.1.1 interview plan and notes; 3.1.2 actors and use-case list                                | HW     |
| <b>3.2</b> | Specification                        | 3.2.1 FR and NFR tables, scenarios (HW); 3.2.2 business rules and data model draft (HD)       | HW, HD |
| <b>4</b>   | <b>Design</b>                        |                                                                                               |        |
| <b>4.1</b> | UML models                           | 4.1.1 use case, class and ER diagrams; 4.1.2 sequence, state and activity diagrams            | HD     |
| <b>4.2</b> | UI design                            | 4.2.1 screen wireframes and prototype screens; 4.2.2 visual design rules                      | MK     |
| <b>5</b>   | <b>Implementation by vibe coding</b> |                                                                                               |        |
| <b>5.1</b> | Prompt engineering                   | 5.1.1 final vibe-coding prompt; 5.1.2 prompt log of iterations against the specification      | HD     |
| <b>5.2</b> | Application build                    | 5.2.1 increment 1 (Must) and increment 2 (Should); 5.2.2 public URL on GitHub Pages           | HD     |
| <b>6</b>   | <b>Testing</b>                       |                                                                                               |        |
| <b>6.1</b> | Unit testing                         | 6.1.1 test cases TC-01x to TC-12x; 6.1.2 passing test run                                     | MK     |
| <b>6.2</b> | Manual QA                            | 6.2.1 scenario walkthrough record and screenshots; 6.2.2 cross-browser and 360/1440 px checks | MK     |
| <b>7</b>   | <b>Final report</b>                  |                                                                                               |        |
| <b>7.1</b> | Content writing                      | 7.1.1 requirements and design chapters (HW); 7.1.2 testing and results chapters (MK)          | HW, MK |
| <b>7.2</b> | Report integration                   | 7.2.1 traceability matrix (HD); 7.2.2 final report PDF, 50–150 pages (MK)                     | HD, MK |

| ID  | Work package        | Deliverables                                                                  | Owner     |
|-----|---------------------|-------------------------------------------------------------------------------|-----------|
| 8   | <b>Presentation</b> |                                                                               |           |
| 8.1 | Slide deck          | 8.1.1 slide outline; 8.1.2 PowerPoint deck                                    | MK        |
| 8.2 | Delivery            | 8.2.1 rehearsed 15-minute talk (MK); 8.2.2 live demo of the deployed app (HD) | MK,<br>HD |

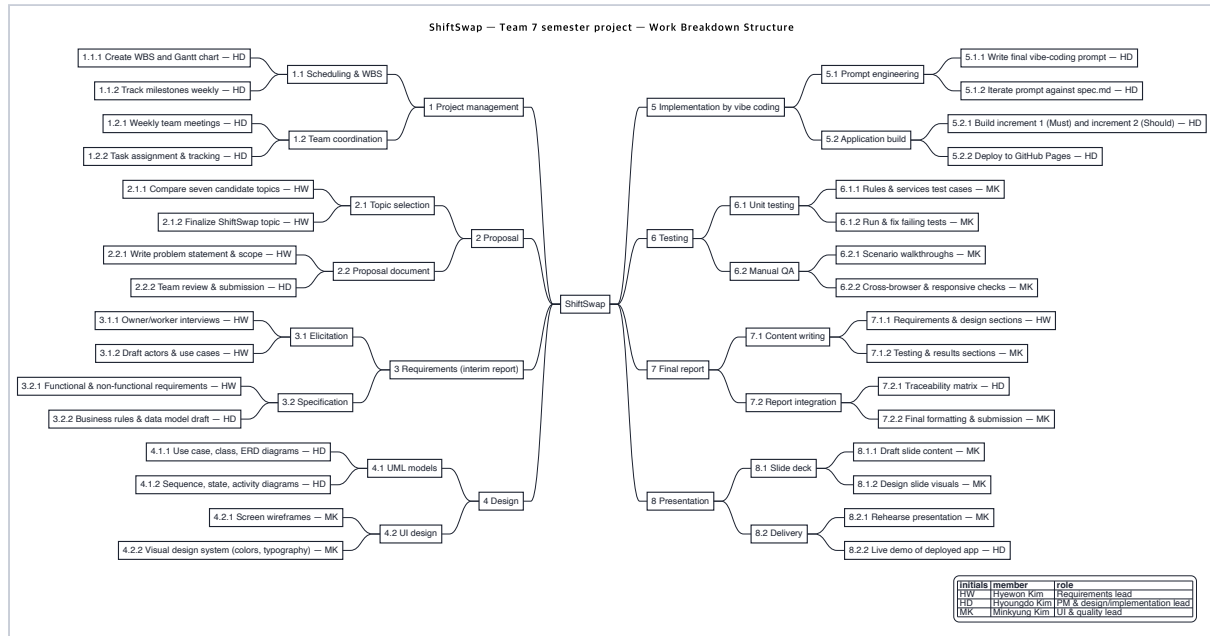


Figure 1. Work breakdown structure of the ShiftSwap term project

## 8. Project schedule

Figure 2 shows the schedule as a Gantt chart. Table 7 gives the same plan week by week. The four course milestones are team formation (Week 3), proposal (Week 5, 2026-10-02), interim report (Week 7, 2026-10-16) and final report with presentation (Week 14, 2026-12-04). Weeks run Monday to Sunday from 2026-08-31 (Week 1); all deadlines are Fridays. Increment 1 is due at the end of Week 10 (2026-11-06) and increment 2 at the end of Week 11 (2026-11-13).

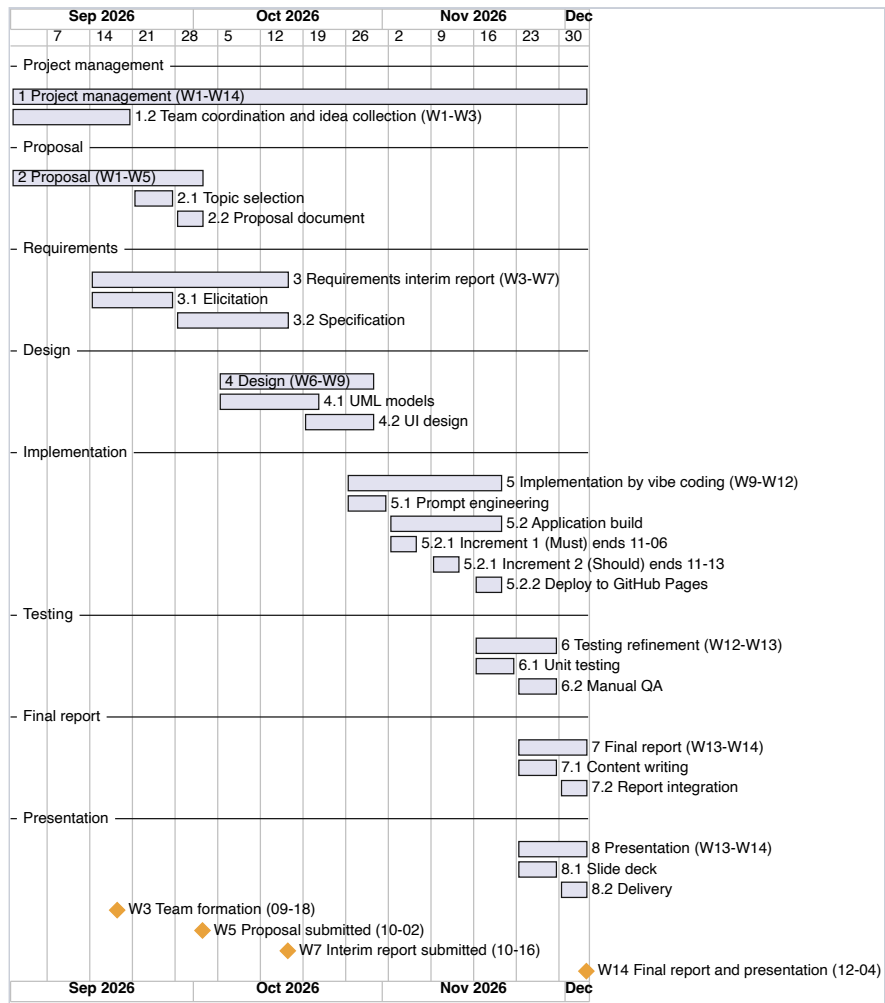


Figure 2. Project schedule (Gantt chart), Week 3 to Week 14

Table 7. Week-by-week schedule (2026)

| Week | Dates         | Work packages                                                                                             | Milestone / output                             |
|------|---------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------|
| 3    | 09-14 – 09-20 | 1.1.1 WBS and Gantt chart                                                                                 | <b>M1: team formed</b> (Fri 2026-09-18)        |
| 4    | 09-21 – 09-27 | 2.1.1 Topic comparison; 2.1.2 Topic decision; 2.2.1 Problem statement and scope; 3.1.1 Interviews begin   | Topic selected (2026-09-27)                    |
| 5    | 09-28 – 10-04 | 2.2.2 Proposal review and submission; 3.1.2 Actors and use cases; 3.2 Specification; 4.1 UML models start | <b>M2: proposal submitted</b> (Fri 2026-10-02) |
| 6    | 10-05 – 10-11 | 3.2.1 FR and NFR; 3.2.2 Business rules and data model; 4.1.1 Use case and class diagrams                  | Requirements draft                             |

| Week | Dates         | Work packages                                                                                                    | Milestone / output                                                |
|------|---------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| 7    | 10-12 – 10-18 | 3.2 Specification complete; 4.1.2 State and activity diagrams; 4.2.1 Screen wireframes start                     | <b>M3: interim report submitted</b> (Fri 2026-10-16)              |
| 8    | 10-19 – 10-25 | 4.1.2 Sequence diagrams; 4.2 UI design                                                                           | UML models complete (10-21)                                       |
| 9    | 10-26 – 11-01 | 4.2.2 Visual design; 5.1 Prompt engineering (5.1.1, 5.1.2)                                                       | UI design complete; prompt version 1                              |
| 10   | 11-02 – 11-08 | 5.2.1 Application build, increment 1 (Must)                                                                      | Increment 1 delivered (Fri 2026-11-06)                            |
| 11   | 11-09 – 11-15 | 5.2.1 Increment 2 (Should)                                                                                       | Increment 2 delivered (Fri 2026-11-13)                            |
| 12   | 11-16 – 11-22 | 5.2.2 Deploy to GitHub Pages; 6.1 Unit testing (6.1.1 test cases, 6.1.2 test runs and fixes)                     | Deployed URL (by 11-20); all test groups run                      |
| 13   | 11-23 – 11-29 | 6.2 Manual QA (6.2.1 scenario walkthroughs, 6.2.2 cross-browser checks); 7.1 Content writing; 8.1 Slide deck     | Final prompt frozen; report draft; slides draft                   |
| 14   | 11-30 – 12-06 | 7.2 Report integration (7.2.1 traceability matrix, 7.2.2 formatting and submission); 8.2 Rehearsal and live demo | <b>M4: final report, slides and presentation</b> (Fri 2026-12-04) |

## 9. Roles of team members

**Table 8. Roles and responsibilities**

| Member       | Student ID | Role                                                                                                                   | Main work packages                    |
|--------------|------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Hyewon Kim   | 24102028   | Requirements lead — problem definition, owner/worker interviews, functional and non-functional requirements, scenarios | 2.1, 2.2.1, 3.1, 3.2.1, 7.1.1         |
| Hyoungdo Kim | 18102075   | Project manager and design/implementation lead — WBS and schedule, UML design models, vibe-coding prompts, deployment  | 1, 2.2.2, 3.2.2, 4.1, 5, 7.2.1, 8.2.2 |
| Minkyung Kim | 23102004   | UI and quality lead — UI screens, test cases, presentation deck, report integration                                    | 4.2, 6, 7.1.2, 7.2.2, 8.1, 8.2.1      |

All three members take part in reviews. The author of a deliverable does not approve it alone: requirements are reviewed by HD and MK, design models by HW and MK, and test cases by HW and HD. The team will meet once a week, and HD will update the Gantt chart after each meeting.

## 10. Risks and mitigation

**Table 9. Project risks**

| ID | Risk                                                                                                                                                                                             | Effect                                                                                  | Mitigation                                                                                                                                                                                                                                                                                                         |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R1 | Legal interpretation of attendance when a shift is handed over. The Labor Standards Act does not state whether a shift given away by agreement breaks perfect attendance for the weekly holiday. | A hard-coded choice could make the holiday allowance wrong for some stores.             | The rule is a store setting (BR-06): EXCUSED (default, the working day was changed by agreement) or ABSENT. Both variants are unit-tested (TC-08x). The report will state the legal sources [3, 5] and that the setting is the owner's decision.                                                                   |
| R2 | Scope creep. Owners may ask for messaging, insurance, tax or leave features.                                                                                                                     | Increment 2 slips past Week 14.                                                         | The Won't list in Section 4 is fixed. A new requirement enters the specification only with a team decision and its priority. Must use cases are delivered first, so the core system is safe even if Should items slip.                                                                                             |
| R3 | Vibe-coding reproducibility. The same prompt can produce different code on a second run.                                                                                                         | The final prompt might not reproduce a working system, which the final report requires. | One specification is the single source of truth for all IDs, names and data. Every prompt version and its result is kept in a prompt log. Acceptance is defined by unit tests (TC-01x to TC-12x) and a scripted walkthrough. In Week 13 the final prompt is re-run in an empty folder and the tests are run again. |
| R4 | Pay rules are implemented incorrectly.                                                                                                                                                           | Wrong amounts on the payroll screen.                                                    | Rules are pure functions separated from the screens. Boundary cases are tested: 14.99 h and 15 h, the 40 h cap, 4 and 5 regular employees, and each probation condition.                                                                                                                                           |
| R5 | Reduced availability of members during the midterm and final exam periods.                                                                                                                       | Weekly tasks slip.                                                                      | No major deliverable is planned in the midterm week. Each work package has one owner and one reviewer who can take over.                                                                                                                                                                                           |

## References

1. Ministry of Employment and Labor, “2026 minimum wage KRW 10,320 per hour,” MOEL notice, Aug. 2025. [https://www.moel.go.kr/news/enews/report/enewsView.do?news\\_seq=18144](https://www.moel.go.kr/news/enews/report/enewsView.do?news_seq=18144)
2. Korea Policy Briefing, “2026년 시간당 최저임금 10,320원.” <https://www.korea.kr/news/policyNewsView.do?newsId=148956812>
3. Labor Standards Act (근로기준법), Art. 11, 18, 55, 56; Enforcement Decree Art. 30. <https://www.law.go.kr>
4. Minimum Wage Act (최저임금법), Art. 5 ②; Enforcement Decree Art. 3. <https://www.law.go.kr>
5. Easy-Law (찾기쉬운 생활법령정보), part-time workers' rest and holidays. <https://easylaw.go.kr/CSP/CnpClsMain.laf?popMenu=ov&csmSeq=896&ccfNo=2&cciNo=3&cnpClsNo=1>
6. D. A. Gustafson, *Schaum's Outline of Software Engineering*, McGraw-Hill, 2002.
7. I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.
8. OMG, *Unified Modeling Language Specification*, v2.5.1, 2017.
9. sql.js — SQLite compiled to WebAssembly. <https://sql.js.org>
10. Shift-scheduling products examined: Shiftee; 알바몬 근무관리; 알바천국 스케줄; When I Work; Homebase; Deputy.